

→ OOPs in java :-

Object oriented programming (OOP) is a programming paradigm based on the concept of objects that contains data (fields) and behavior (methods).

It is used to :

- Improves code reusability
- Enhances maintainability and scalability
- Makes programs easier to understand and manage.
- Closely models real-world entities.

Characteristics of OOPs :-

Class	A blueprint or template for creating objects.
Object	An instance of class.
Encapsulation	Protects data from direct access.
Abstraction	Hides unnecessary details.
Polymorphism	Allows the same method to behave differently.
Inheritance	Reuses code from existing classes.

→ Class and object in java :-
Class and object are the **basic building blocks** of **OOP** in java.

1. What is a class?

A class is a blueprint, template, or design from which objects are created ~~that~~ that share common properties and behavior (method).

Eg;

```
class Student {
```

```
    int age;
```

```
    String name;
```

```
    void display() {
```

```
        System.out.println("Name=" + name);
```

```
        System.out.println("Age=" + age);
```

```
    }
```

```
}
```

here:

Student → Class

name → Data member

age → Data member

display() → method

∴ The Class itself is only a **blueprint**. We create objects from it.

2. What is an object?

An object is an **instance of a class**.

An object represents an actual entity and has:

- State
- behavior
- Identity

Eg;

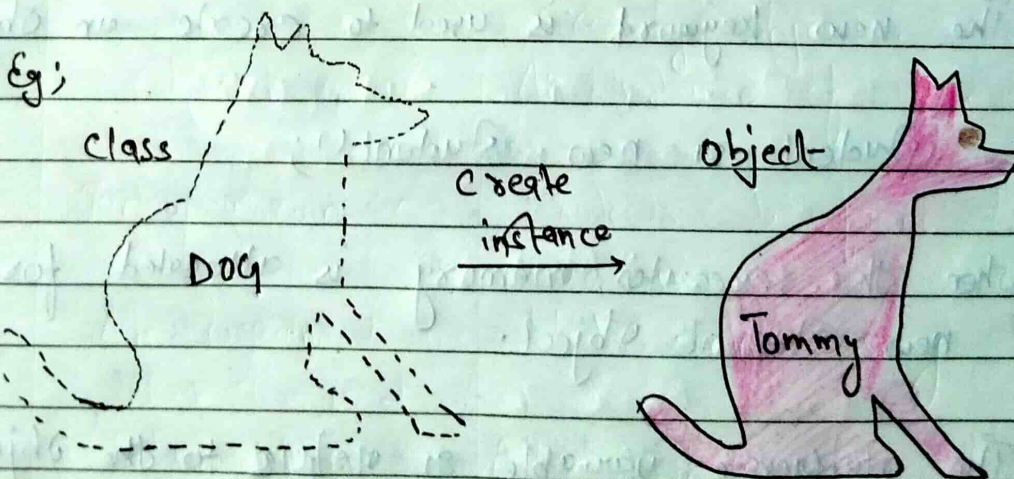
`Student s1 = new Student();`

here;

`Student` → Class

`s1` → Reference variable

`new Student()` → Object



Properties	Method	Property value	methods
Name	<code>getName()</code>	Name: Tommy	<code>getName()</code>
colour	<code>getColour()</code>	Colour: Pink	<code>getName()</code>
Eye-colour	<code>getEyeColour()</code>	Eye colour: Brown	<code>getEyeColour()</code>
Height	<code>getHeight()</code>	Height: 17 inch	<code>getHeight()</code>
length	<code>Come Here()</code>	length: 35 inch	<code>get Here()</code>

→ Creating a class :

Syntax :

```
class classname {  
    // variable, methods  
}
```

→ Creating an object :

Syntax :

```
ClassName referenceVariable = new ClassName();
```

• What does new do?

The new keyword is used to create an object.

```
Student s1 = new Student();
```

When this executes, memory is allocated for the new Student object.

The reference variable s1 refers to the object

• Accessing Data member?
 We use the dot (.) operator to access member of an object.

Syntax:
 object.variable

• Calling a method using an object:
 We also use the dot operator to call methods.

Syntax:
 object.method();

→ Complete example : Class & object :-

```
class Student { // class Student contain data member and
    int age;      // methods
    String name; // data member
    void display () { // methods
        System.out.println("name=" + name);
        System.out.println("age=" + age);
    }
}

public class pogo { // Main class
    public static void main (String args[]) {
        Student s1 = new Student(); // creating an object
        s1.age = 21;
        s1.name = "Akash";
        s1.display(); // calling the display() method using the obj
    }
}
```

→ Ex for multiple objects:→

```
Class Student {
```

```
    int age;
```

```
    String name;
```

```
    void display() {
```

```
        System.out.println("name = " + name);
```

```
        System.out.println("age = " + age);
```

```
    }
```

```
}
```

```
public class Pogo {
```

```
    public static void main (String args[]) {
```

```
        Student s1 = new Student();
```

```
        Student s2 = new Student();
```

```
        s1.name = "Akash";
```

```
        s1.age = 22;
```

```
        s2.name = "Ayush";
```

```
        s2.age = 21;
```

```
        s1.display();
```

```
        s2.display();
```

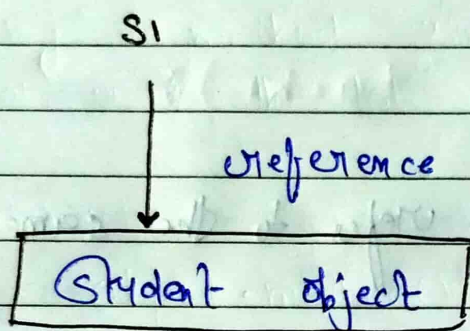
```
    }
```

```
}
```

→ Reference variable

```
Student s1 = new Student();
```

s1 is a **reference variable** that refers to the object.

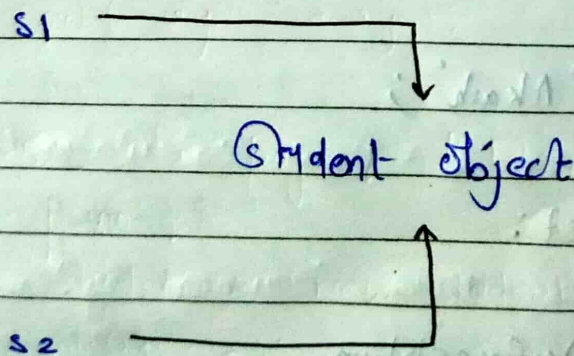


→ Multiple reference variables to an object :-

```
Student s1 = new Student();
```

```
Student s2 = s1;
```

Now both references refer the **same object**.



if we write :

`s1.name = "Akash";`

then

`cout (< s2.name);`

also prints

Akash

because s1 and s2 refer to the same object.

→ Null reference :-

A reference can contain null.

`Student s1 = null;`

This means :

s1 → no object

if we try :

`s1.name = "Akash";`

we will get :

NullPointerException

because s1 doesn't refer to an object

→

Anonymous Object :-

An object without a reference variable is called a anonymous object.

Eg;

```
new Student().display();
```

here, the object has no reference such as st.

it is generally useful when the object is needed only once.

Eg;

```
class Student {
```

```
    void display() {
```

```
        cout << "Hello Student";
```

```
    }
```

```
}
```

```
public class pogo {
```

```
    psvm {
```

```
new new Student().display();
```

```
}
```

```
}
```

methods and

→ Class with multiple object : // User input
 import java.util scanner;
 class calculator {

int a, b;

void add () {

 cout ("sum=" + (a+b));

}

void multiple () {

 cout ("product=" + (a*b));

}

}

public class pogo {

 psum {

 Calculator c = new calculator ();

 Scanner sc = new Scanner (System.in);

 cout ("enter two num1;

 c.a = sc.nextInt(); // taking input

 c.b = sc.nextInt();

c.add();

e.multiply();

}

}

~~Not~~